

**Sans documents excepté une copie double resto-verso avec notes manuscrites**

Les paramètres des fonctions sont parfois volontairement omis, à vous de les définir.

La clarté de vos réponses sera prise en compte.

N'oubliez pas de commenter vos programmes

**Vous aurez besoin de 3 copies : une par partie**

## **Partie 1 : Gestion d'une bibliothèque universitaire (7 points) -> Copie n°1**

### **Base de données, fichiers CSV**

Pour l'organisation et le suivi de ses emprunts de ses livres par les étudiants, la bibliothèque souhaite créer une base de données composée des tables décrites dans le modèle relationnel ci-dessous.

```
Etudiant(#id_etudiant: int, nom: str, prenom: str, email: str) avec email unique, et nom
et prenom non nul
Auteur(#id_auteur: int, nom: str, prenom: str) avec nom et prenom non nul
Livre(#id_livre: int, code_livre: str, titre: str, annee: int, auteur=>Auteur) avec
code_livre unique, et titre non nul
Emprunt(#id_etudiant=>Etudiant, #id_livre=>Livre)
```

Les données des livres sont enregistrées dans un fichier CSV intitulé « livres.csv ». Le séparateur est le point-virgule (;). Le fichier contient une ligne d'en-tête.

#### **Exemple de contenu :**

```
code;titre;prenom_auteur;nom_auteur;année
L001;Le Petit Prince;Antoine;de Saint-Exupéry;1943
L002;1984;George;Orwell;1949
L003;Dune;Frank;Herbert;1965
L004;Fondation;Isaac;Asimov;1951
L005;L'Étranger;Albert;Camus;1942
L006;Les Misérables;Victor;Hugo;1862
L007;Fahrenheit 451;Ray;Bradbury;1953
```

- 1.1 Écrire la requête SQL permettant de créer la table `Livre` du modèle relationnel ci-dessus. N'oubliez pas d'ajouter toutes les contraintes mentionnées.
- 1.2 On suppose que la base de données SQLite `bibliotheque.sqlite` a déjà été créée et remplie par les données des auteurs, mais pas avec celles des livres. Écrire la fonction Python `importer_livres(connecteur, nom_fichier: str)` qui lit le fichier CSV dont le nom est passé en paramètre et insère automatiquement chaque livre dans la table `Livre` de la base de données dont le connecteur est passé en paramètre. Au choix, vous utiliserez le module CSV ou le module Pandas. On supposera que tous les auteurs mentionnés dans le fichier, sont connus de la base de données.
- 1.3 Écrire la fonction `livres_empruntes_par(connecteur, nom: str, prenom: str) -> list`. Cette fonction doit exécuter une requête SQL pour interroger la base de données dont le connecteur est passé en paramètre. L'objectif est de récupérer les livres empruntés par l'étudiant dont le nom et le prénom sont fournis. La fonction doit retourner uniquement une liste de titres (ex: ["Le Petit Prince", "Dune"]).
- 1.4 Écrire le programme principal qui ajoute les livres du fichier « livres.csv » dans la table `Livre` de la base de données `bibliotheque.sqlite` et qui demande à l'utilisateur le nom et le prénom d'un étudiant et affiche la liste des titres des livres empruntés par cet étudiant. On traitera les exceptions.
- 1.5 Comment modifier le modèle relationnel ci-dessus de telle manière qu'on puisse intégrer le fait qu'un livre puisse être écrit par plusieurs auteurs ?

### Partie 3 : Il fait chaud ! (6 points) -> Copie n°3

Dans cette partie, on exploitera lorsque c'est possible, les possibilités offertes par numpy pour les `ndarray` (méthodes et fonctions). Un exemple est donné en début d'énoncé mais le code doit être générique et fonctionner pour d'autres exemples.

#### ndarray, courbe, interface graphique

Une station météorologique enregistre chaque jour les températures (en °C) de plusieurs villes dont les noms sont donnés dans un tableau (ndarray 1D). Les températures sont stockées dans un tableau (ndarray 2D). Chaque ligne représente une ville et chaque colonne un jour.

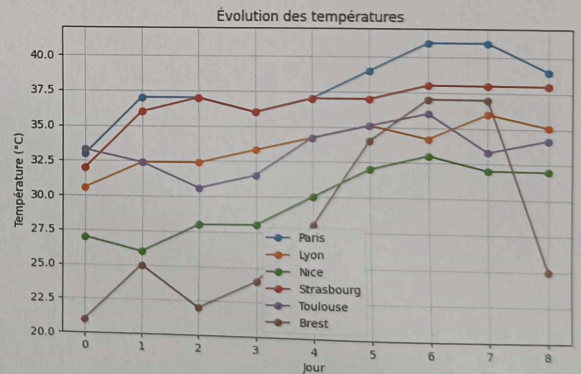
Les tableaux suivants donnent un exemple pour 6 villes et sur 9 jours.

```
import numpy as np
import matplotlib.pyplot as plt

villes = np.array(["Paris", "Lyon", "Nice", "Strasbourg", "Toulouse", "Brest"])

temperatures = np.array([
    [33, 37, 37, 36, 37, 39, 41, 41, 39], ← ville
    [34, 36, 36, 37, 38, 39, 38, 40, 39],
    [27, 26, 28, 28, 30, 32, 33, 32, 32],
    [32, 36, 37, 36, 37, 37, 38, 38, 38],
    [37, 36, 34, 35, 38, 39, 40, 37, 38],
    [21, 25, 22, 24, 28, 34, 36, 36, 25]
])
```

- 3.1 Écrire la fonction `plus_grand_ecart(temperatures, villes)` qui retourne le nom de ville qui a le plus grand écart entre la température minimale et maximale observée sur la période.
- 3.2 Écrire la fonction `jour_plus_chaud(temperature)` qui calcule la moyenne des températures par jour et retourne la valeur et le numéro du jour le plus chaud sur la période.
- 3.3 Écrire la fonction `affiche_courbes(temperatures, villes)` qui affiche sur une seule courbe l'évolution des températures sur la période pour toutes les villes d'un tableau de températures (voir un exemple ci-contre)
- 3.4 Certaines villes prévoient de faire des travaux d'urbanisme visant à réduire la température de leur ville de 10%. La liste `villes_travaux` contient le nom des villes concernées. Écrire une fonction `simulation(temperatures, villes, villes_travaux)` qui calcule et retourne le tableau des températures en appliquant la réduction pour les villes concernées.
- 3.5 Soit l'interface graphique définie partiellement comme le montre le code ci-dessous. Écrire la fonction `callback affiche_plus_chaud()` qui, à l'aide des fonctions précédentes, affiche un message dans le label prévu à cet effet. Le message donne le jour le plus chaud et sa température moyenne après avoir appliqué la simulation des travaux d'urbanisme pour les villes listées dans l'attribut `_villes_travaux`.



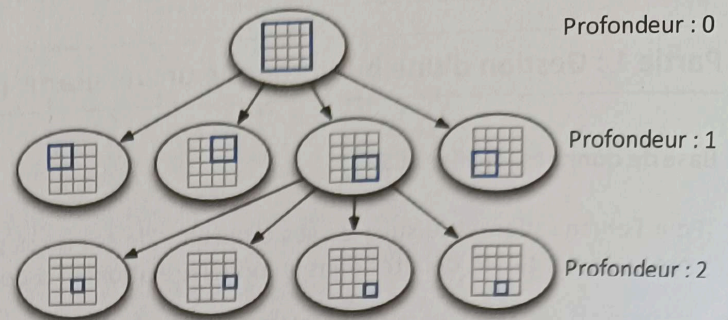
```
class Fenetre(tk.Tk):
    def __init__(self, temperatures, villes):
        tk.Tk.__init__(self)
        self.title('evolution des temperatures')
        self.temperatures = temperatures # temperatures relevées
        self.villes = villes # villes pour les temperatures relevées
        self.villes_travaux = ["Paris", "Toulouse", "Lyon"] # villes ayant
        self.message = tk.StringVar()
        tk.Label(self, textvariable=self.message).grid(row = 2, column=1, columnspan = 2)
        tk.Button(self, text=' jour plus chaud',
            command=self.affiche_plus_chaud).grid(row=3, column=1)
```

## Partie 2 : Codage d'images binaires (7 points) -> Copie n°2

### Objet, ndarray et images

Le dernier drone de l'ESA s'est posé avec succès sur Mars et est prêt à renvoyer vers la Terre les dernières images de la surface. Cependant, en raison de difficultés techniques imprévues, le drone ne peut envoyer que des images binaires. Une image binaire est ici un tableau 2D de valeurs booléennes.

Afin de faciliter la transmission de ces images binaires, il faudrait que le drone puisse les compresser à l'aide d'un « Quadtree ». Votre tâche dans le cadre de cette nouvelle mise à jour du logiciel consiste à concevoir un programme en Python pour transformer une image en image binaire puis encoder l'image binaire dans une structure arborescente qui permettra au Quadtree la compression. La structure arborescente est composée de nœuds (voir la figure ci-contre) que vous allez coder en suivant les étapes décrites ci-dessous.



#### 2.1 Créez une classe ImageBinaire ayant les méthodes suivantes :

- Un **constructeur** qui accepte comme paramètre un tableau (ndarray) contenant une image couleur (3D) ou en niveaux de gris (2D) ou binaire (2D). S'il ne s'agit pas d'un ndarray, on déclenche une exception. Si l'image n'est pas binaire, le constructeur transforme l'image en une image binaire et la met dans l'attribut `_data`. La transformation d'une image en niveaux de gris en image binaire peut se faire grâce à un seuillage (masque booléen) qu'on peut fixer ici à la valeur 128 (on suppose le codage des pixels entre 0 et 255). Une transformation d'image couleur en image binaire consiste à transformer d'abord l'image couleur en niveau de gris (en mettant un poids de 1/3 à chaque canal RVB par exemple) puis à transformer l'image de niveau de gris en image binaire avec la méthode décrite ci-dessus.
- Une méthode `divisible()` qui renvoie une valeur booléenne indiquant si l'image enregistrée dans l'attribut `_data` peut être divisée en quatre images plus petites. Autrement dit, si sa largeur et sa hauteur sont >1.
- Une méthode `unicolor()` qui renvoie « True » si tous les éléments de `_data` sont identiques, et « False » si au moins l'un d'entre eux est différent. Vous pouvez utiliser la fonction `np.unique()` vue en cours.
- Une méthode `diviser()` qui divise l'image contenue dans l'attribut `_data` en quatre sous-images binaires, de tailles égales, à plus ou moins 1 pixel près. La méthode renvoie la liste de ces quatre images (binaires).
- Une méthode `afficher()` qui affiche l'attribut `_data` sous forme d'image à l'aide de matplotlib (False : noir, True : blanc)

#### 2.2 Créez une classe NoeudBinaire qui est une classe dérivée de ImageBinaire, avec les fonctions suivantes :

- Un **constructeur** qui accepte deux paramètres : un entier appelé *profondeur* dont la valeur par défaut est 0 et une image. Le constructeur doit :
  - Appeler le constructeur de la classe ImageBinaire avec le paramètre pour initialiser l'attribut `_data` avec l'image passée en paramètre.
  - Incrémenter de 1 le compteur de l'attribut de classe `_nb_noeuds`, qui sert à comptabiliser le nombre total de nœuds créés.
  - Créer un attribut `_prof` qui indique la profondeur par rapport au nœud racine. Il se verra attribuer la valeur du paramètre *profondeur*.
  - Créer un attribut `_fils` qui est une liste éventuellement vide des nœuds fils. Si l'image contenue dans l'attribut `_data` n'est pas monochrome et peut être divisée, le constructeur la découpe en quatre images plus petites à l'aide de la méthode `diviser()` et les utilise pour créer quatre nœuds fils distincts dont la profondeur est augmentée de 1. Ces nœuds fils sont eux-mêmes des NoeudBinaire et sont insérés dans une liste et assignés à l'attribut `_fils`.

**Remarque :** monochrome signifie que tous les pixels ont la même valeur

- Une méthode appelée `max_prof()` qui permet de déterminer la profondeur maximale d'un objet NoeudBinaire. Dans l'exemple de la figure ci-dessus, si on appelle la méthode `max_prof()` pour le nœud racine, elle retournera la valeur 2.